DSA + JAVA LAST 10 MIN REVISION SHEET

1. ARRAY BASICS

Add Element

```
arr[count] = value;
count++;
```

Display Array

```
for(int i=0; i<count; i++){
    System.out.print(arr[i] + " ");
}
```

---

2. INSERT ELEMENT AT POSITION

Logic

Shift RIGHT → insert value.

Example

Before:

```
10 20 30 40
```

Insert 25 at pos=2

After:

```
10 20 25 30 40
```

Code

```
for(int i=count; i>pos; i--){
    arr[i] = arr[i-1];
}

arr[pos] = val;
count++;
```

---

3. DELETE BY POSITION

Logic

Shift LEFT.

Code

```
for(int i=pos; i<count-1; i++){
    arr[i] = arr[i+1];
}
count--;
```

---

4. DELETE BY VALUE

Code

```
int vpos = -1;

for(int i=0; i<count; i++){
    if(arr[i] == val){
        vpos = i;
        break;
    }
}

if(vpos != -1){

    for(int i=vpos; i<count-1; i++){
```

```
        arr[i] = arr[i+1];
    }

    count--;
}
```

---

5. REVERSE ARRAY

Logic

Swap start/end.

Code

```
int start = 0;
int end = count-1;

while(start < end){

    int temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;

    start++;
    end--;
}
```

---

6. LEFT ROTATE ARRAY

Code

```
int temp = arr[0];

for(int i=0; i<count-1; i++){
    arr[i] = arr[i+1];
}

arr[count-1] = temp;
```

7. RIGHT ROTATE ARRAY

Code

```
int temp = arr[count-1];

for(int i=count-1; i>0; i--){
    arr[i] = arr[i-1];
}

arr[0] = temp;
```

8. SEQUENTIAL SEARCH

Logic

Check one-by-one.

Code

```
int pos = -1;

for(int i=0; i<count; i++){

    if(arr[i] == val){
        pos = i;
        break;
    }
}
```

Time Complexity: O(n)

9. BINARY SEARCH (NON-RECURSIVE)

IMPORTANT

Array MUST be sorted.

Logic

middle bigger → LEFT middle smaller → RIGHT

Code

```
int start = 0;
int end = count-1;
int pos = -1;

while(start <= end){

    int mid = (start + end)/2;

    if(arr[mid] == val){
        pos = mid;
        break;
    }

    else if(val > arr[mid]){
        start = mid + 1;
    }

    else{
        end = mid - 1;
    }
}
```

Time Complexity: O(log n)

---

10. RECURSIVE BINARY SEARCH

Code

```
static int binarySearch(int arr[], int start, int end, int key){

    if(start > end){
        return -1;
    }

    int mid = (start + end)/2;
```

```
    if(arr[mid] == key){
        return mid;
    }

    else if(key > arr[mid]){
        return binarySearch(arr, mid+1, end, key);
    }

    else{
        return binarySearch(arr, start, mid-1, key);
    }
}
```

---

11. 2D ARRAY INPUT

Code

```
Scanner sc = new Scanner(System.in);

int rows = sc.nextInt();
int cols = sc.nextInt();

int arr[][] = new int[rows][cols];

for(int i=0; i<rows; i++){

    for(int j=0; j<cols; j++){

        arr[i][j] = sc.nextInt();
    }
}
```

---

12. DISPLAY 2D ARRAY

Code

```
for(int i=0; i<rows; i++){

    for(int j=0; j<cols; j++){
```

```
            System.out.print(arr[i][j] + " ");
        }

        System.out.println();
    }
```

---

13. ROW SUM

Code

```
for(int i=0; i<rows; i++){

    int sum = 0;

    for(int j=0; j<cols; j++){

        sum = sum + arr[i][j];
    }

    System.out.println(sum);
}
```

---

14. COLUMN SUM

Code

```
for(int j=0; j<cols; j++){

    int sum = 0;

    for(int i=0; i<rows; i++){

        sum = sum + arr[i][j];
    }

    System.out.println(sum);
}
```

---

15. TRANSPOSE MATRIX

Logic

arr[i][j] → arr[j][i]

Code

```
for(int i=0; i<cols; i++){

    for(int j=0; j<rows; j++){

        System.out.print(arr[j][i] + " ");
    }

    System.out.println();
}
```

---

16. IDENTITY MATRIX

Logic

if(i==j) → 1 else → 0

Code

```
boolean identity = true;

for(int i=0; i<rows; i++){

    for(int j=0; j<cols; j++){

        if(i == j){

            if(arr[i][j] != 1){
                identity = false;
            }
        }

        else{
```

```
            if(arr[i][j] != 0){
                identity = false;
            }
        }
    }
}
```

---

17. BUBBLE SORT

Logic

Compare neighbors. Largest element reaches end every pass.

Code

```
for(int i=0; i<count-1; i++){

    for(int j=0; j<count-1-i; j++){

        if(arr[j] > arr[j+1]){

            int temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}
```

Time Complexity: O(n^2)

---

18. SELECTION SORT

Logic

Find minimum and place at beginning.

Code

```
for(int i=0; i<count-1; i++){

    int min = i;

    for(int j=i+1; j<count; j++){

        if(arr[j] < arr[min]){
            min = j;
        }
    }

    int temp = arr[i];
    arr[i] = arr[min];
    arr[min] = temp;
}
```

---

19. INSERTION SORT

Logic

Pick element and insert in correct place.

Code

```
for(int i=1; i<count; i++){

    int key = arr[i];
    int j = i-1;

    while(j>=0 && arr[j] > key){

        arr[j+1] = arr[j];
        j--;
    }

    arr[j+1] = key;
}
```

---

20. STACK USING ARRAY

LIFO

Last In First Out

Important

push → insert pop → remove

Code

```
int top = -1;

// PUSH
if(top == capacity-1){
    System.out.println("Overflow");
}
else{
    top++;
    arr[top] = val;
}

// POP
if(top == -1){
    System.out.println("Underflow");
}
else{
    System.out.println(arr[top]);
    top--;
}
```

---

21. QUEUE USING ARRAY

FIFO

First In First Out

Important Variables

front = 0 rear = -1

ENQUEUE

```
rear++;
arr[rear] = val;
```

DEQUEUE

```
System.out.println(arr[front]);
front++;
```

---

22. LINKED LIST BASICS

Node Structure

```
class Node{

    int data;
    Node next;

    Node(int data){
        this.data = data;
        this.next = null;
    }
}
```

---

23. ADD NODE END

Code

```
Node newNode = new Node(val);

if(head == null){
    head = newNode;
}

else{
```

```
    Node temp = head;

    while(temp.next != null){
        temp = temp.next;
    }

    temp.next = newNode;
}
```

---

24. DISPLAY LINKED LIST

Code

```
Node temp = head;

while(temp != null){

    System.out.print(temp.data + " ");

    temp = temp.next;
}
```

---

25. TREE TRAVERSALS

Inorder

LEFT ROOT RIGHT

Preorder

ROOT LEFT RIGHT

Postorder

LEFT RIGHT ROOT

---

26. BST INSERT

Logic

smaller → left bigger → right

---

27. GRAPH

BFS

Uses Queue

DFS

Uses Stack/Recursion

---

28. TIME COMPLEXITY SHORT NOTES

O(1)

Constant

O(n)

Linear

O(log n)

Binary Search

O(n^2)

Nested loops / Bubble Sort

---

29. RECURSION

IMPORTANT

Function calls itself.

MUST HAVE

Base condition.

Example:

```
if(n==0)
```

Otherwise infinite calls.

---

30. CALL BY VALUE

Copy passed. Original value unchanged.

---

31. CALL BY REFERENCE

Address/reference passed. Original value changes.

FINAL EXAM SURVIVAL TIPS

1. Write logic first.
2. Use meaningful variable names.
3. Don’t panic on syntax.
4. Array + loops solve many questions.
5. Nested loops = 2D arrays.
6. Stack = top variable.
7. Queue = front/rear.
8. Binary Search requires sorted array.
9. Bubble sort = neighbor swap.
10. Insertion sort = shifting.